

Use Case 3: Movie Ratings Analytics Warehouse

Batch Processing with Snowflake + PySpark + AWS Glue

1. Overview

StreamFlix, a growing movie streaming platform, needs to build a **star schema data warehouse** to power its analytics dashboard. The raw data arrives in mixed formats: the movie catalog and user profiles are delivered as CSV files, while the ratings data (generated by the platform's recommendation engine) is stored as Apache Parquet files.

Your job is to design and implement an ETL pipeline using **AWS Glue (PySpark)** that: - Ingests data from multiple file formats (CSV and Parquet) - Performs data quality validation - Transforms the data into a dimensional star schema - Loads the results into **Snowflake** for downstream analytics - Creates analytical views for the business intelligence team

2. Business Scenario

StreamFlix's analytics team has identified the following business questions they need answered:

1. **What are the top-rated movies by genre?** -- The content acquisition team needs this to inform licensing decisions.
2. **How do rating patterns vary by user demographics?** -- The marketing team wants to target campaigns by age group and country.
3. **What are the monthly rating trends?** -- The product team wants to understand engagement over time.
4. **Which genres are most popular across different regions?** -- The recommendation engine team needs this for model tuning.

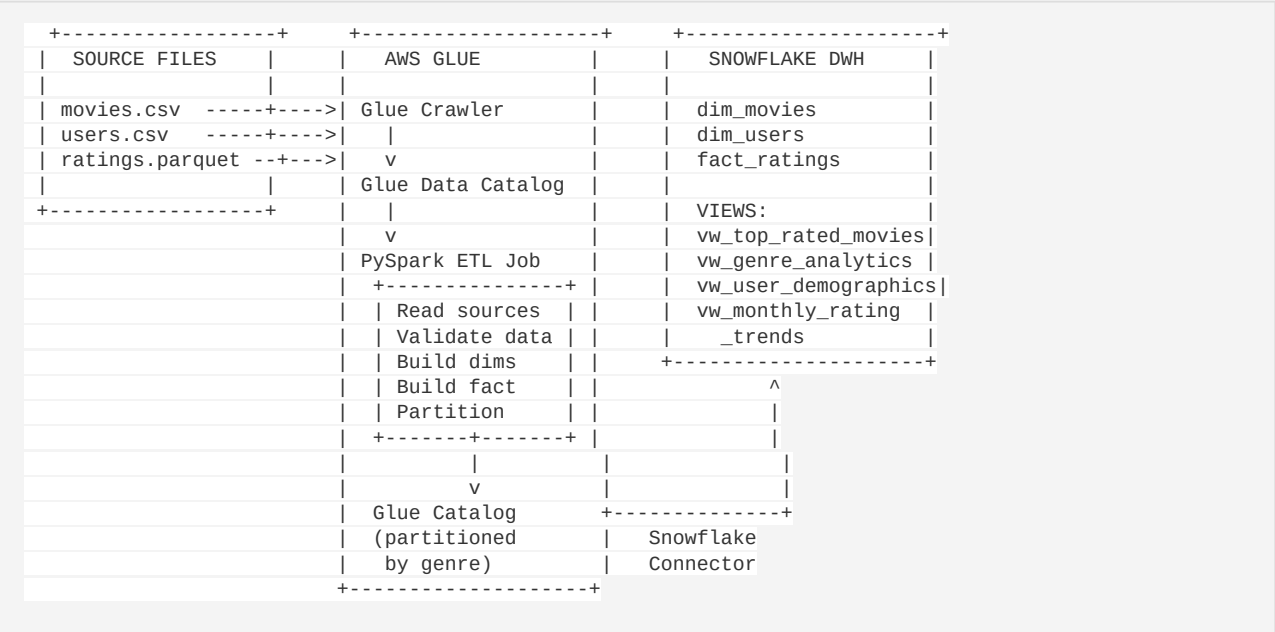
Currently, the data sits in separate files with no unified schema. Analysts are writing ad-hoc scripts that produce inconsistent results. A proper data warehouse with a star schema will provide a single source of truth.

3. Learning Objectives

By completing this use case, you will be able to:

- **Handle Parquet files** in PySpark and understand why columnar formats matter for analytics
 - **Design a star schema** with fact and dimension tables, including surrogate keys
 - **Implement data quality checks** (range validation, foreign key integrity, null handling)
 - **Use AWS Glue** to crawl, catalog, and partition data
 - **Load transformed data into Snowflake** using the Spark-Snowflake connector
 - **Create Snowflake views** (standard and materialized) for analytical queries
 - **Partition data** in the Glue Data Catalog for optimized query performance
-

4. Architecture Diagram



Data Flow: 1. Source files (CSV + Parquet) are stored in S3 (or local for development). 2. A Glue Crawler discovers the schema and registers tables in the Glue Data Catalog. 3. A PySpark Glue Job reads from the catalog, validates data, builds the star schema. 4. Transformed data is written back to the Glue Catalog (partitioned by genre). 5. The star schema tables are loaded into Snowflake via the Spark-Snowflake connector. 6. Analytical views are created in Snowflake via JDBC.

5. Dataset Description

5.1 movies.csv (Movie Catalog)

Column	Type	Description	Example
movie_id	String	Unique movie identifier	MOV001
title	String	Movie title	The Last Frontier
genre	String	Primary genre	Action
release_year	Integer	Year of release	2019
director	String	Director name	James Harlow
duration_min	Integer	Runtime in minutes	132

rating_pg	String	Parental guidance rating	PG-13
-----------	--------	--------------------------	-------

50 movies across 8 genres: Action, Comedy, Drama, Sci-Fi, Horror, Romance, Thriller, Animation.

5.2 users.csv (User Profiles)

Column	Type	Description	Example
user_id	String	Unique user identifier	USR001
username	String	Display name	cinephile_max
age	Integer	User age	28
gender	String	M or F	M
country	String	Country of residence	USA
registration_date	Date	Account creation date	2020-03-15

80 users from 8 countries: USA, UK, Canada, India, Australia, Germany, France, Japan.

5.3 ratings.parquet (User Ratings)

Column	Type	Description	Example
rating_id	String	Unique rating identifier	RAT0001
user_id	String	Foreign key to users	USR001
movie_id	String	Foreign key to movies	MOV023
rating	Float64	Rating score (1.0 to 5.0, step 0.5)	4.5
review_text	String	Optional text review (nullable)	"Absolutely loved it"
timestamp	Timestamp	When the rating was submitted	2023-07-15 14:30:22

~500 ratings with approximately 30% null reviews.

5.4 Why Parquet?

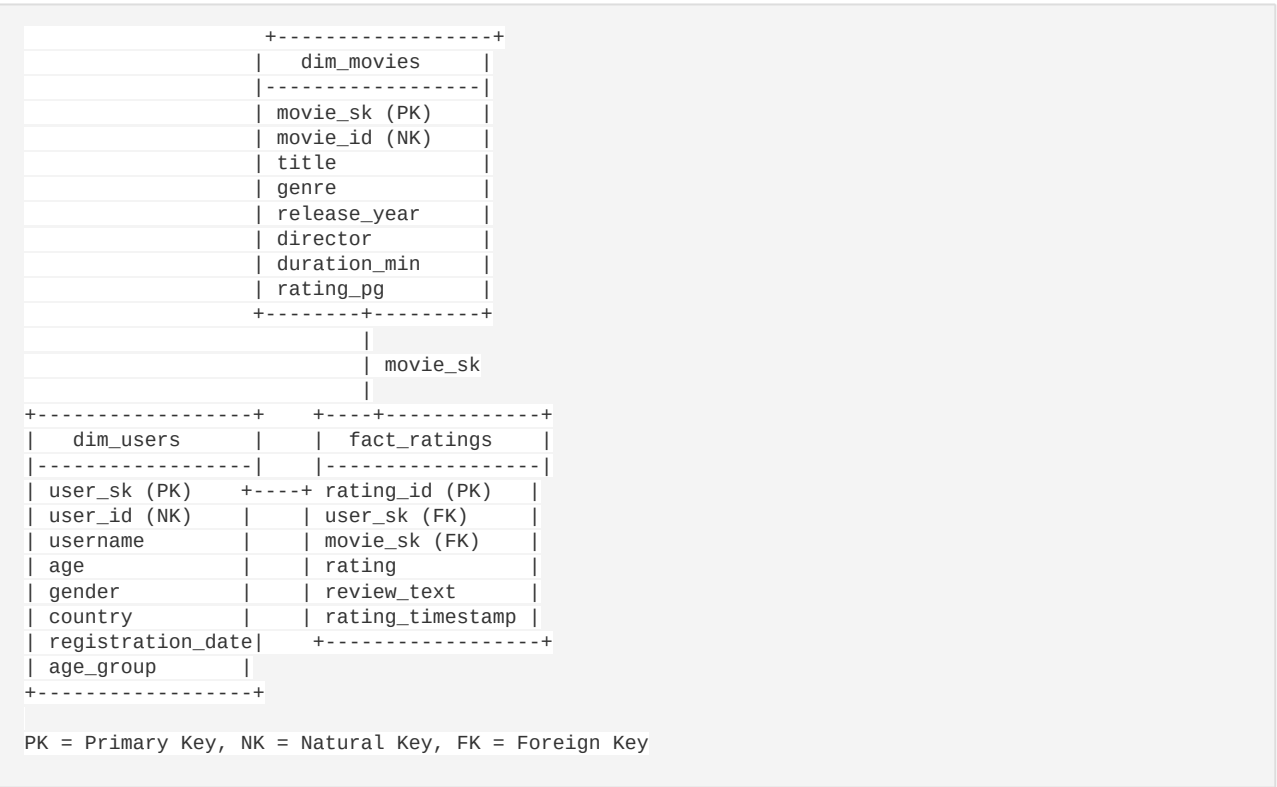
Apache Parquet is a **columnar storage format** widely used in big data ecosystems. Key advantages:

- **Columnar compression:** Only reads the columns you need, reducing I/O significantly
- **Schema embedded:** The file carries its own schema -- no need for external definitions
- **Predicate pushdown:** Query engines can skip entire row groups that don't match filters
- **Type-rich:** Supports timestamps, decimals, nested types -- unlike CSV
- **Ecosystem support:** Native support in Spark, Glue, Athena, Snowflake, and more

In this use case, the ratings data uses Parquet because it is the highest-volume table and benefits most from columnar compression and efficient reads.

6. Star Schema Design

The target data warehouse follows a **star schema** pattern:



Key design decisions: - **Surrogate keys** (movie_sk, user_sk) are integer-based for join performance - **Natural keys** (movie_id, user_id) are preserved for traceability - **Derived columns:** age_group is calculated in the dimension (e.g., "18-24", "25-34") - The **fact table** is kept narrow -- only keys, the measure (rating), and descriptive fields

7. Tasks

Task 1: Generate the Ratings Dataset

Run the provided Python script to create the Parquet file.

```
cd datasets/use_case_3/
pip install pyarrow pandas
python generate_parquet.py
```

Verify: Confirm that ratings.parquet has been created. Optionally inspect it:

```
import pyarrow.parquet as pq
table = pq.read_table("datasets/use_case_3/ratings.parquet")
print(table.schema)
print(f"Rows: {table.num_rows}")
```

Task 2: Set Up Snowflake Star Schema

Create the target tables in Snowflake using the DDL provided in the solution (or write your own).

Requirements: - Database: STREAMFLIX_DWH - Schema: ANALYTICS - Tables: dim_movies, dim_users, fact_ratings - Surrogate keys use AUTOINCREMENT or IDENTITY - Appropriate data types and NOT NULL constraints

Task 3: Read CSV and Parquet Data in PySpark

Write a PySpark script (or Glue job) that reads all three source files:

```
# Read CSVs with header and schema inference
movies_df = spark.read.csv("s3://bucket/use_case_3/movies.csv", header=True, inferSchema=True)
users_df = spark.read.csv("s3://bucket/use_case_3/users.csv", header=True, inferSchema=True)

# Read Parquet (schema is embedded)
ratings_df = spark.read.parquet("s3://bucket/use_case_3/ratings.parquet")
```

Verify: Print schemas and row counts for all three DataFrames.

Task 4: Data Quality Checks

Implement the following validations:

1. **Rating range:** Ensure all ratings are between 1.0 and 5.0 (inclusive)
2. **Valid increments:** Ratings should be in 0.5 increments only
3. **Foreign key integrity:** Every user_id in ratings exists in users; every movie_id in ratings exists in movies
4. **Null checks:** rating_id, user_id, movie_id, and rating must not be null
5. **Duplicate check:** No duplicate rating_id values

Log any violations and decide whether to filter or fail.

Task 5: Build Dimension Tables

Transform the source data into dimension tables:

dim_movies: - Add a surrogate key (movie_sk) using monotonically_increasing_id() - Retain all original columns - Ensure no duplicate movie_id

dim_users: - Add a surrogate key (user_sk) using `monotonically_increasing_id()` - Derive age_group column: - 16-17 -> "Under 18" - 18-24 -> "18-24" - 25-34 -> "25-34" - 35-44 -> "35-44" - 45-54 -> "45-54" - 55-64 -> "55-64" - 65+ -> "65+" - Retain all original columns

Task 6: Build the Fact Table

Create fact_ratings by joining ratings with dimension tables:

1. Join ratings_df with dim_movies on movie_id to get movie_sk
 2. Join the result with dim_users on user_id to get user_sk
 3. Select: rating_id, user_sk, movie_sk, rating, review_text, timestamp (renamed to rating_timestamp)
 4. Drop the natural keys from the final fact table
-

Task 7: Register in Glue Catalog with Partitioning

Write the transformed data to S3 and register in the Glue Data Catalog:

- **dim_movies:** Partitioned by genre
- **dim_users:** Partitioned by country
- **fact_ratings:** No partitioning (or partition by year if desired)

Use `awsglue.context.GlueContext.write_dynamic_frame` or `spark.write.partitionBy()`.

Task 8: Load to Snowflake

Use the Spark-Snowflake connector to write the star schema tables:

```
sf_options = {
    "sfURL": "<account>.snowflakecomputing.com",
    "sfUser": "<user>",
    "sfPassword": "<password>",
    "sfDatabase": "STREAMFLIX_DWH",
    "sfSchema": "ANALYTICS",
    "sfWarehouse": "COMPUTE_WH",
}

dim_movies_df.write \
    .format("net.snowflake.spark.snowflake") \
    .options(**sf_options) \
    .option("dbtable", "DIM_MOVIES") \
    .mode("overwrite") \
    .save()
```

Repeat for dim_users and fact_ratings.

Task 9: Create Snowflake Analytical Views

After loading, create the following views in Snowflake:

1. **vw_top_rated_movies:** Movies with average rating, rating count, and rank within genre
2. **vw_genre_analytics:** Aggregate stats by genre (avg rating, total ratings, unique users)

3. `vw_user_demographics`: Rating behavior grouped by `age_group` and country
4. `vw_monthly_rating_trends`: Monthly aggregates showing rating volume and average over time

8. Expected Outputs

After completing all tasks, you should have:

Artifact	Location	Description
<code>ratings.parquet</code>	<code>datasets/use_case_3/</code>	Generated Parquet file (~500 rows)
Snowflake <code>dim_movies</code>	<code>STREAMFLIX_DWH.ANALYTICS</code>	50 rows with surrogate keys
Snowflake <code>dim_users</code>	<code>STREAMFLIX_DWH.ANALYTICS</code>	80 rows with <code>age_group</code> derived column
Snowflake <code>fact_ratings</code>	<code>STREAMFLIX_DWH.ANALYTICS</code>	~500 rows with surrogate FK references
Glue Catalog tables	<code>streamflix_db</code>	3 tables, <code>dim_movies</code> partitioned
4 Snowflake views	<code>STREAMFLIX_DWH.ANALYTICS</code>	Analytical views for BI team

Verification counts: - `dim_movies`: 50 rows, 8 distinct genres - `dim_users`: 80 rows, 8 distinct countries, 7 distinct age groups - `fact_ratings`: ~500 rows, all `user_sk` values exist in `dim_users`, all `movie_sk` values exist in `dim_movies`

9. Hints

Level 1: Gentle Nudge

- For reading Parquet in PySpark, you do not need to specify a schema -- it is embedded in the file.
- `monotonically_increasing_id()` generates unique IDs, but they are not sequential. That is fine for surrogate keys.
- When checking foreign key integrity, use a `left_anti` join to find orphaned records.
- The Spark-Snowflake connector uses `format("net.snowflake.spark.snowflake")`.

Level 2: More Specific

- To derive `age_group`, use PySpark's `when().otherwise()` chain:

```
python from pyspark.sql.functions import when, col df = df.withColumn("age_group", when(col("age") < 18, "Under 18") .when(col("age") <= 24, "18-24") .when(col("age") <= 34, "25-34") # ... continue for other groups .otherwise("65+") )
```
- To validate rating increments, check that `(rating * 2) % 1 == 0`.

- For the fact table join, use two sequential inner joins: `python fact_df = ratings_df \ .join(dim_movies_df.select("movie_id", "movie_sk"), "movie_id") \ .join(dim_users_df.select("user_id", "user_sk"), "user_id")`

Level 3: Almost the Answer

- Complete dimension build: `python from pyspark.sql.functions import monotonically_increasing_id`

```
dim_movies_df = movies_df.withColumn("movie_sk", monotonically_increasing_id()) dim_users_df =
users_df.withColumn("user_sk", monotonically_increasing_id()) \ .withColumn("age_group", when(col("age") < 18, "Under 18")
.when(col("age") <= 24, "18-24") .when(col("age") <= 34, "25-34") .when(col("age") <= 44, "35-44") .when(col("age") <= 54,
"45-54") .when(col("age") <= 64, "55-64") .otherwise("65+")) - For Glue catalog partitioning:python
dim_movies_df.write.partitionBy("genre").mode("overwrite") \ .option("path", "s3://bucket/warehouse/dim_movies/") \
.saveAsTable("streamflix_db.dim_movies") `` - Snowflake view for top rated movies usesAVG(), COUNT(), andRANK()
OVER (PARTITION BY ...).
```

10. Validation Queries

Run these queries in Snowflake after the pipeline completes to verify correctness:

```
-- 1. Row counts
SELECT 'dim_movies' AS tbl, COUNT(*) AS cnt FROM ANALYTICS.DIM_MOVIES
UNION ALL
SELECT 'dim_users', COUNT(*) FROM ANALYTICS.DIM_USERS
UNION ALL
SELECT 'fact_ratings', COUNT(*) FROM ANALYTICS.FACT_RATINGS;

-- 2. Referential integrity: orphaned facts (should return 0 rows)
SELECT f.rating_id
FROM ANALYTICS.FACT_RATINGS f
LEFT JOIN ANALYTICS.DIM_MOVIES m ON f.movie_sk = m.movie_sk
WHERE m.movie_sk IS NULL;

SELECT f.rating_id
FROM ANALYTICS.FACT_RATINGS f
LEFT JOIN ANALYTICS.DIM_USERS u ON f.user_sk = u.user_sk
WHERE u.user_sk IS NULL;

-- 3. Rating range check (should return 0 rows)
SELECT * FROM ANALYTICS.FACT_RATINGS
WHERE rating < 1.0 OR rating > 5.0;

-- 4. Distinct genres in dim_movies (should be 8)
SELECT COUNT(DISTINCT genre) AS genre_count FROM ANALYTICS.DIM_MOVIES;

-- 5. Age group distribution
SELECT age_group, COUNT(*) AS user_count
FROM ANALYTICS.DIM_USERS
GROUP BY age_group
ORDER BY age_group;

-- 6. Top 5 movies by average rating (minimum 5 ratings)
SELECT m.title, m.genre, AVG(f.rating) AS avg_rating, COUNT(*) AS num_ratings
FROM ANALYTICS.FACT_RATINGS f
JOIN ANALYTICS.DIM_MOVIES m ON f.movie_sk = m.movie_sk
GROUP BY m.title, m.genre
HAVING COUNT(*) >= 5
ORDER BY avg_rating DESC
LIMIT 5;

-- 7. Genre analytics summary
SELECT m.genre,
COUNT(*) AS total_ratings,
ROUND(AVG(f.rating), 2) AS avg_rating,
COUNT(DISTINCT f.user_sk) AS unique_users
FROM ANALYTICS.FACT_RATINGS f
JOIN ANALYTICS.DIM_MOVIES m ON f.movie_sk = m.movie_sk
```



```
GROUP BY m.genre
ORDER BY total_ratings DESC;

-- 8. Monthly rating trend
SELECT DATE_TRUNC('MONTH', f.rating_timestamp) AS rating_month,
       COUNT(*) AS total_ratings,
       ROUND(AVG(f.rating), 2) AS avg_rating
FROM ANALYTICS.FACT_RATINGS f
GROUP BY rating_month
ORDER BY rating_month;
```

Appendix: File Locations

File	Path
Movie catalog	datasets/use_case_3/movies.csv
User profiles	datasets/use_case_3/users.csv
Parquet generator	datasets/use_case_3/generate_parquet.py
Generated ratings	datasets/use_case_3/ratings.parquet
Snowflake DDL	solutions/use_case_3/snowflake_setup.sql
Glue ETL job	solutions/use_case_3/glue_job.py
Solution walkthrough	solutions/use_case_3/README.md